Universidade de Brasília
Departamento de Ciência da Computação
CIC0101 · Sistemas de Informação

2026 / 2

Ficha 1: Construção do MER

Professora: Patrícia Araújo de Oliveira

Compilado: 2 de novembro de 2026

Grupo 1 — Integrantes

Luan Moura	1111111111	Leonardo	22222222
Calebe	33333333	Maíra	44444444

Sumário

1 Entidades, atributos e identificadores 1

2 Mathematical Foundations 2

3 Algorithms 2

4 Code 2

5 Figures and Plots 4

6 Exercises 4

1 Entidades, atributos e identificadores

Uma entidade representa um assunto sobre qual o sistema precisa registrar informações. O identificador distingue uma ocorrência dos demais. A primeira linha é apenas um exemplo.

2 Mathematical Foundations

Definition 2.1: Big-O notation

Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We write $f(n) = \mathcal{O}(g(n))$ if there exist $c > 0$ and $n_0 \in \mathbb{N}$ such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$.

Theorem 2.2: Merge sort recurrence

Let $T(n) = 2T(n/2) + n$ with $T(1) = 1$ for n a power of two. Then $T(n) = n \log_2 n + n = \Theta(n \log n)$.

Entidade	**Atributos necessários**	**Identificador**	**RF relacionado(s) RF01 (ilustrativo)**
Solicitação	número, data de abertura, situação	número de solicitação	RF01 (ilustrativo)

Demonstração. We argue by induction on k where $n = 2^k$. For $k = 0$, $T(1) = 1 = 1 \cdot 0 + 1$. Assume the claim for $n/2$. Then

$$\begin{aligned} T(n) &= 2\,T(n/2) + n \\ &= 2\left(\tfrac{n}{2}\log_2 \tfrac{n}{2} + \tfrac{n}{2}\right) + n \\ &= n(\log_2 n - 1) + n + n \\ &= n\log_2 n + n. \end{aligned}$$

■

Example 2.3: Gradient descent step

For a differentiable $f : \mathbb{R}^d \to \mathbb{R}$ the update with step size $\eta > 0$ is

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta\,\nabla f(\boldsymbol{x}_t), \qquad \boldsymbol{x}^\star \in \arg\min_{\boldsymbol{x}} f(\boldsymbol{x}).$$

Note. Use `\begin{theorem}{Title}{label}` and refer to it with `Theorem \ref{thm:label}`. For instance, Theorem 2.2 above.

A small table with `booktabs`:

Tabela 1: Common running times.

Algorithm	**Time**	**Space**
Merge sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Quicksort (avg)	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$
Dijkstra (heap)	$\mathcal{O}((V + E) \log V)$	$\mathcal{O}(V)$

Algorithm 1 Dijkstra's shortest paths

Input: Graph $G = (V, E)$, weights $w : E \to \mathbb{R}_{\geq 0}$, source $s \in V$
Output: Shortest-path distances $d[v]$ from s to every $v \in V$

$d[v] \leftarrow \infty$ for all $v \in V$; $d[s] \leftarrow 0$
$Q \leftarrow$ priority queue containing all $v \in V$ keyed by $d[v]$
while $Q \neq \emptyset$ **do**
    $u \leftarrow \textsc{ExtractMin}(Q)$
    for all edges $(u, v) \in E$ **do**
        if $d[u] + w(u, v) < d[v]$ **then**
            $d[v] \leftarrow d[u] + w(u, v)$
            $\textsc{DecreaseKey}(Q, v, d[v])$
return d

3 Algorithms

> **Lemma 3.1: Correctness invariant**
>
> When a vertex u is extracted from Q, $d[u]$ equals the true shortest-path distance $\delta(s, u)$.

4 Code

```python
def merge_sort(a):
    """Return a sorted copy of list `a` in O(n log n) time."""
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):   # merge step
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]
```

Listing 1: Merge sort in Python.

```cpp
#include <vector>

// Returns the index of x in sorted vector a, or -1 if absent.
int binary_search(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x)  lo = mid + 1;
        else             hi = mid - 1;
    }
    return -1;
}
```

Listing 2: Binary search in C++.

Pitfall. Writing `(lo + hi) / 2` can overflow for large indices; use `lo + (hi - lo) / 2` as in Listing 2.

5 Figures and Plots

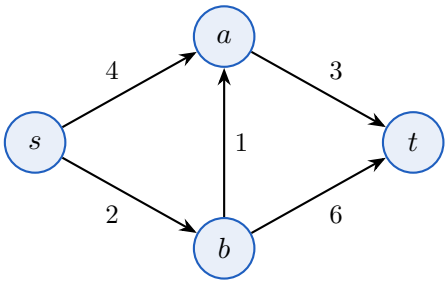


Figura 1: A small weighted digraph. The shortest s–t path is $s \to b \to a \to t$ with cost 6.

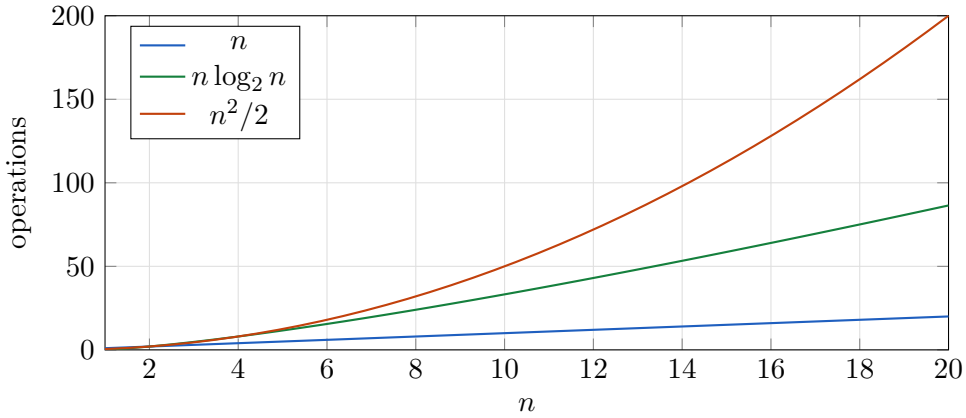


Figura 2: Growth rates of common running times.

6 Exercises

Exercise 6.1: Recurrence

Solve $T(n) = 3T(n/2) + n$ with $T(1) = 1$ using a recursion tree, and give a tight Θ-bound.

Exercise 6.2: Implementation

Modify Listing 1 to count the number of inversions in $\mathcal{O}(n \log n)$ time.

Further Reading

- T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms*, MIT Press.

- J. Kleinberg, É. Tardos. *Algorithm Design*, Pearson.